



Vita Save Decryptor (native)

Project report & user guide — how a PS Vita save decryptor was rebuilt
from scratch, fully offline

v0.2.0-native

Windows x64 · single .exe

100% offline — no key service

Byte-exact output

Made by **The New Game+**

[YouTube](#) · [Ko-fi](#) · [Discord](#)

Source: github.com/Johnzx07/vita-save-decryptor

01 The app in one minute

What it is, why it exists, and what makes this version different.

Vita Save Decryptor (native) turns a PS Vita CMA save backup (`savedata.psvimg`) into plain files ready for the Vita3K emulator. It is a single Windows executable — no install, no setup.

0

ONLINE SERVICES NEEDED

1

RUNTIME DEPENDENCY (PYCRYPTODOME)

4/4

ENCRYPTED FILES BYTE-EXACT VS REFERENCE

~10s

TYPICAL DECRYPT TIME PER SAVE

The problem this solves

The original tooling for CMA backups required an external program (`psvpfparser.exe`) that phoned home to the F00D key service at `cma.henkaku.xyz` . If that service goes down — or you're offline — decryption stops working. This project reverse-engineered the entire PFS encryption scheme and re-implemented it in pure Python, so keys are now derived **locally from the save's own sealed key**.

BOTTOM LINE

Same output as the original tool — verified byte-for-byte — with zero network access and no bundled parser executable.

Contents

- 01 The app in one minute
- 02 How to use the app — step by step
- 03 Troubleshooting & FAQ
- 04 How it was done — the reverse-engineering story
- 05 Technical reference — keys, XTS-AES & the ICV oracle
- 06 Credits, licenses & support

02 How to use the app

From backup folder to decrypted save in four steps.

1

Get your CMA backup + key

On a PS Vita with the CMA plugin, back up the game's savedata. You'll get a folder containing `savedata.psvimg` — and the site shows you a **64-character hex key**. Write it down; you need it for every backup.

2

Launch VitaSaveDecryptor.exe

Double-click the executable (from the GitHub release). No installation, no admin rights needed.

3

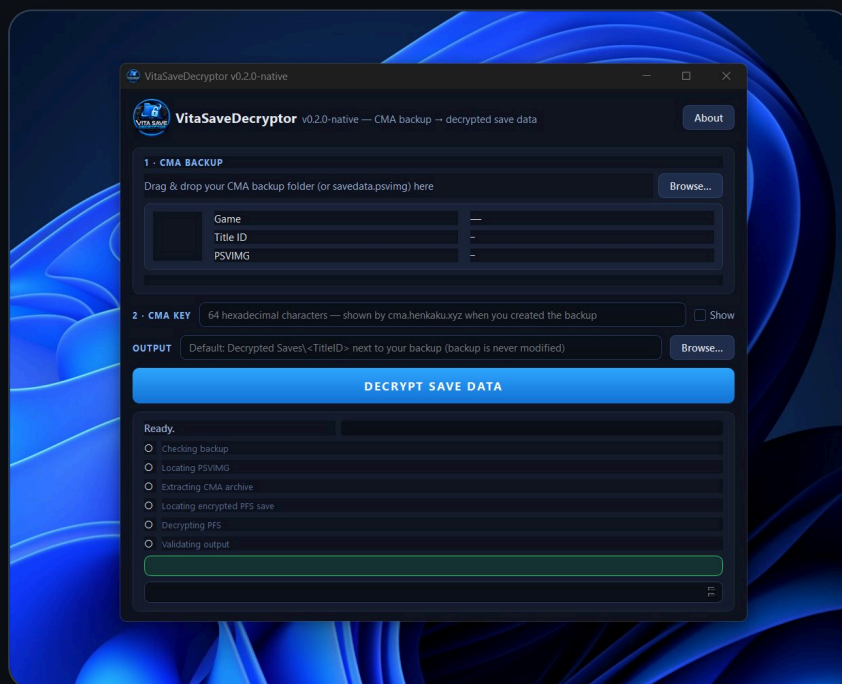
Point it at your backup & enter the key

Click **Browse...** and select either the `.psvimg` file itself or any folder that contains it. Paste the 64-hex CMA key into the field.

4

Choose an output folder & hit Decrypt

Pick where decrypted files should go, then click **Decrypt Save....** Watch the six pipeline stages complete — your save is ready for Vita3K.



The main window — pick a backup, enter the CMA key, choose output, decrypt.

GOOD TO KNOW

Your original backup is **never modified**. Everything is written to the separate output folder you chose. Re-running on the same backup always produces identical results (the process is fully deterministic).

03 Output & troubleshooting

What you get, and what to do when something goes wrong.

What the output looks like

The decrypted set mirrors the game's PFS savedata layout:

```
your_output_folder/
├─ system.dat           ← save data (encrypted → now plain)
├─ sce_sys/
│   ├─ param.sfo        ← metadata (title, size, version...)
│   ├─ sdslot.dat        ← slot info
│   ├─ safemem.dat       ← safe memory
│   ├─ keystone          ← Henkaku keystone (if present)
│   └─ sealedkey         ← the key material used for decryption
```

Load this folder into Vita3K as a save file and you're done.

Common problems

SYMPTOM	CAUSE & FIX
"Invalid CMA key"	The key must be exactly 64 hex characters. Copy it fresh from the CMA site — no spaces, no quotes.
"CMA extraction failed (stage 1)"	Wrong CMA key for this backup, or a corrupted <code>.psvimg</code> . Each backup has its own unique key.
"No PFS save set found"	The archive isn't a Vita savedata. Make sure you backed up the game's <i>savedata</i> , not the whole app package.
"PFS decryption failed (stage 2)"	The sealed key didn't match this save — usually means the backup and key are from different backups. Pair them correctly.
Nothing happens on launch	Windows SmartScreen may block unsigned exes: click <i>More info</i> → <i>Run anyway</i> . Or check your antivirus quarantine.

NOTE

The app is fully offline, but the CMA key itself comes from the henkaku.xyz CMA site when you create a backup. If you lose your key, you'll need to re-back up (or find it in your browser history) — there's no way around that; it's how CMA works.

Requirements

- ▶ Windows 10 or 11, 64-bit
- ▶ A PS Vita CMA save backup (`savedata.psvimg`) + its 64-hex key
- ▶ No internet connection required at any point during decryption

04 How it was done

The reverse-engineering story — from a dead-end dependency to a byte-exact pure-Python decryptor.

The goal: remove the online key service. That meant answering one question — *how does psvpfsparser turn a sealed key into per-file decryption keys?* The answer required pulling apart the reference tool's C++ source and rebuilding its math from scratch.

P1

Map the pipeline & capture ground truth

reconnaissance

Documented the two-stage flow (CMA unpack → PFS decrypt), captured a real h-encore release save (PCSG90096) as a regression fixture, and recorded the reference tool's exact output byte-for-byte — the target to beat.

P2

Read the reference implementation (read-only)

source analysis

Pulled psvpfstools' C++ sources locally and traced the call path: `xts_dec` → `pfs_decrypt_icv` → `XTS-AES`. Confirmed the cipher is **XTS-AES-128 with two derived keys**, 0x8000-byte sectors, IEEE-1619 perturbation that carries across all 512 sub-blocks per sector.

P3

Crack the klicensee — the breakthrough

key derivation

The per-file keys derive from a 16-byte "klicensee" value. Brute-forcing candidate derivations of the sealed key, one stood out: **AES-CBC decrypting the first 16 bytes of the sealed-key blob with a fixed PSVita system key** — producing exactly the value the F00D service had returned. All four encrypted files then matched byte-for-byte.

P4

Which files decrypt, which copy?

classification

Parsed the set's `files.db` (SCENGPFS v3) and found a clean rule in the type field: `type & 0x4000` → **copy verbatim, otherwise XTS-decrypt**. Verified against every file in the fixture — no hardcoded file lists.

04 The ICV Merkle oracle

The final piece — resolving which salt belongs to which file, with zero guessing.

Each encrypted file is keyed by a per-file **salt** (a 32-bit value stored as the name of its `.icv` record). The set contains several salts and no direct "file → salt" mapping — so how do you know which one to use? Heuristics (entropy, text-likeness) worked for most files but failed on small binaries like `keystone`, where three candidate salts all looked plausible.

THE INSIGHT

The stored 20-byte ICV in each record is a Merkle root computed over the file's **ciphertext** sectors — not the plaintext. That makes it an oracle: for every candidate salt, compute the expected root and compare. Exactly one matches per file. Deterministic, no guessing.

Verification results

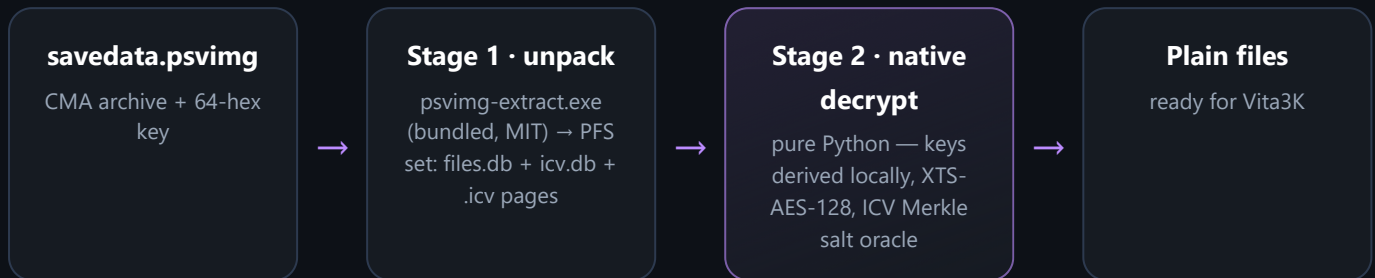
FILE	TYPE RULE	SALT (ICV ORACLE)	RESULT VS REFERENCE TOOL
<code>system.dat</code>	XTS-decrypt	<code>aad62c72</code>	✓ byte-exact (105,472 B)
<code>sce_sys/sdslot.dat</code>	XTS-decrypt	<code>58b8e8cf</code>	✓ byte-exact (46,080 B)
<code>sce_sys/safemem.dat</code>	XTS-decrypt	<code>621b0519</code>	✓ byte-exact (16,384 B)
<code>sce_sys/keystone</code>	XTS-decrypt	<code>cbab98f3</code>	✓ byte-exact (1,024 B)
<code>sce_sys/param.sfo</code>	copy verbatim	—	✓ identical
<code>sce_sys/sealedkey</code>	copy verbatim	—	✓ identical

A regression test suite (`tests/test_pfs_native.py`) re-runs this end-to-end on the vendored fixture — 9 checks, all passing — so any future change is immediately checked against ground truth.

05 Technical reference

The exact math, for anyone who wants to reproduce it.

Pipeline



Key derivation chain

```
# 1. klicensee - AES-CBC decrypt of the sealed-key blob's first block
klicensee = AES-CBC-Decrypt( sealedkey_blob[0:16], key=PSVITA_FIXED_KEY, iv=zeros )

# 2. per-file keys (ctr selects data key vs tweak key)
base0 = SHA1( klicensee )
saltin = salt.to_bytes(4,"little") + ctr.to_bytes(4,"little") # 8 bytes
key_ctr = SHA1( base0 || SHA1(saltin).digest() ).digest()[ :16 ]
dec_key, tweak_enc_key = key_ctr(ctr=1), key_ctr(ctr=2)

# 3. ICV secret (for the Merkle oracle)
secret = SHA1( SHA1(klicensee) || SHA1( [0xA, salt]_LE8 ) )
```

XTS-AES-128 sector decryption

```
# per 0x8000-byte sector: seed once, perturb across all 512 sub-blocks
seed = AES-ECB( tweak_enc_key, u64_LE( 0x8000 * sector_index ) )
for each 16-byte block i in the sector (i = 0..511):
    tweak_i = IEEE-1619-perturb( seed, i ) # GF(2^128) × x, carry across blocks
    plain[i] = AES-ECB-Decrypt( dec_key, cipher[i] XOR tweak_i )
```

05 The salt oracle & constants

How the right salt is found — and every magic number in one place.

ICV Merkle tree (salt oracle)

```
# leaf per sector: HMAC-SHA1 over the CIPHERTEXT sector
leaf_j = HMAC-SHA1( secret, ciphertext_sector_j )
# complete binary tree, BFS heap layout (1-based; children of i are 2i, 2i+1)
root   = merkle_fold( leaves )                # left-inherits / right-advances
match  = ( root == stored_icv_root_in_.icv_record ) # exactly one salt matches per file
```

Constants & magic numbers

CONSTANT	VALUE / RULE
PSVITA_FIXED_KEY	16-byte fixed key (in source) — AES-CBC-decrypts the sealed-key blob's first block to recover <code>klicensee</code>
Sector size	<code>0x8000</code> bytes = 512×16 -byte sub-blocks; XTS seed advances per sector
Cipher mode	XTS-AES-128, two keys — data key (<code>ctr=1</code>) + tweak key (<code>ctr=2</code>), IEEE-1619 perturbation across all 512 blocks
Per-file salt	32-bit LE; <code>key = SHA1(SHA1(klicensee) SHA1(salt ctr))[:16]</code>
ICV record header	<code>nSectors</code> @ offset <code>0x28</code> ; stored ICV root (HMAC-SHA1) @ offset <code>0x2C</code>
files.db type rule	<code>type & 0x4000</code> → copy verbatim, otherwise XTS-decrypt

DESIGN NOTE

No code was copied from the reference tool — it served strictly as a behavioral specification. The implementation is original Python (pycryptodome for AES/SHA1), verified against captured ground truth rather than by diffing source.

06 Credits & support

Credit where it's due — and how to reach the project.

Built by

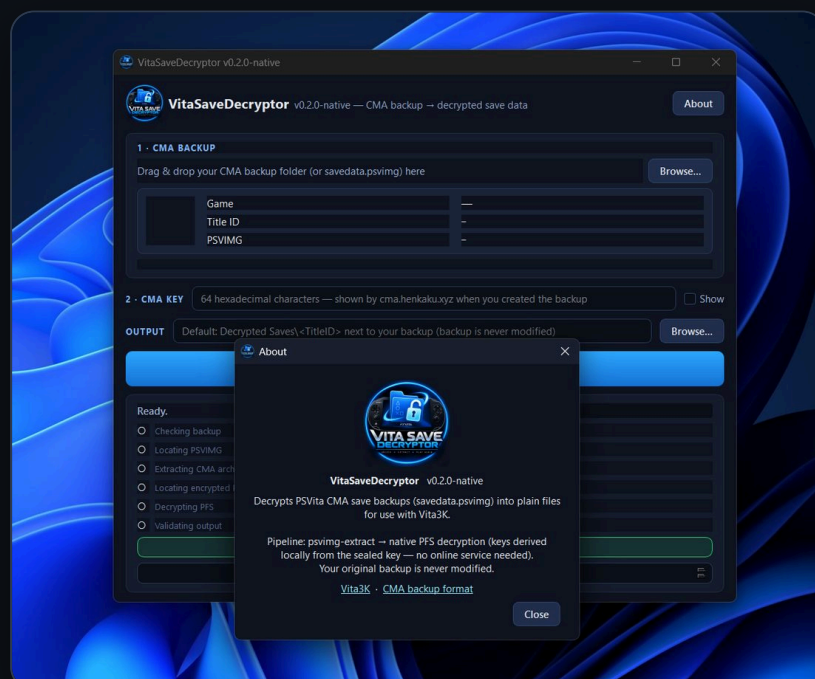
The New Game+ — author, GUI, and native re-implementation. This report was written as part of the open-source release at github.com/Johnzx07/vita-save-decryptor.

Third-party components & licenses

COMPONENT	LICENSE	ROLE IN THIS PROJECT
Vita Save Decryptor (this app)	MIT — © 2026 The New Game+	The application itself
psvimgtools — Yifan Lu	MIT (© 2017)	Bundled stage-1 CMA unpacker + Cygwin DLLs
PySide6 — The Qt Company & contributors	LGPL v3 / Commercial	GUI toolkit (dynamically linked, unmodified)
pycryptodome — Helder Eijs	BSD-3-Clause	AES / SHA1 primitives for stage 2
h-encore — TheFloW	MIT (© 2018)	Public regression test fixture (PCSG90096) only
psvpfstools — motoharu-gosuto	No license published upstream	Read-only behavioral reference (nothing copied or redistributed)

The Vita3K community

This work builds on PS Vita preservation research from the broader community, including the [Vita3K](#) emulator project and its PFS tooling lineage. No Vita3K source is compiled into this app — the credit reflects research, compatibility work, and community guidance.



The About dialog in-app carries the same links.

Support & community



youtube.com/@TheNewGamePluss — guides & walkthroughs



ko-fi.com/thenewgameplus — buy me a coffee



NGP "Bench" community — hang out & get help

LICENSE

This application is released under the MIT License. Full third-party notices live in `THIRD_PARTY_NOTICES.md` and `licenses/` in the repository.